

08 — Constraints in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Constraint Overview](#)
 3. [NOT NULL](#)
 4. [PRIMARY KEY](#)
 5. [UNIQUE](#)
 6. [FOREIGN KEY](#)
 7. [CHECK](#)
 8. [EXCLUSION Constraints](#)
 9. [Deferrable Constraints](#)
 10. [Managing Constraints \(ALTER TABLE\)](#)
 11. [Constraint Violations and Error Handling](#)
 12. [SQL Examples](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Apply all six constraint types correctly
 - Configure foreign key cascade behavior
 - Use deferrable constraints for complex data loads
 - Add and drop constraints on live tables safely
 - Understand how constraints affect query planning and performance
-

Constraint Overview

Constraint Types

— NOT NULL	– column may not be NULL
— PRIMARY KEY	– unique, not null identifier for each row
— UNIQUE	– no two rows have the same value (NULL excepted)
— FOREIGN KEY	– referential integrity to another table
— CHECK	– arbitrary boolean expression must be true
— EXCLUSION	– generalized uniqueness using operators (GIST/GIN)

Scope:

Column constraint	– defined inline on a single column
Table constraint	– defined separately, can span multiple columns

NOT NULL

```

-- Column constraint
CREATE TABLE users (
    user_id    BIGINT NOT NULL,
    email      TEXT   NOT NULL,
    full_name  TEXT,      -- nullable (no NOT NULL)
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Adding NOT NULL later
ALTER TABLE users ALTER COLUMN full_name SET NOT NULL;
-- WARNING: fails if any existing rows have NULL full_name
-- First fix the data:
UPDATE users SET full_name = 'Unknown' WHERE full_name IS NULL;
ALTER TABLE users ALTER COLUMN full_name SET NOT NULL;

-- Removing NOT NULL
ALTER TABLE users ALTER COLUMN full_name DROP NOT NULL;

```

How NOT NULL interacts with DEFAULT:

```

-- DEFAULT is applied before NOT NULL check
-- So this is valid: no NOT NULL violation at insert time
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
-- INSERT without created_at uses the default (not NULL)

```

PRIMARY KEY

A PRIMARY KEY is syntactic sugar for: `NOT NULL + UNIQUE` .

```

-- Single-column PK (column constraint)
CREATE TABLE users (
    user_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email   TEXT   NOT NULL
);

-- Single-column PK (table constraint)
CREATE TABLE users (
    user_id BIGINT GENERATED ALWAYS AS IDENTITY,
    email   TEXT   NOT NULL,
    PRIMARY KEY (user_id)
);

-- Composite PK (table constraint only)
CREATE TABLE order_items (
    order_id    INTEGER NOT NULL,
    product_id  INTEGER NOT NULL,
    quantity    INTEGER NOT NULL,
    unit_price  NUMERIC(10,4) NOT NULL,
    PRIMARY KEY (order_id, product_id)

```

```
);

-- Named PK
CREATE TABLE products (
    product_id INTEGER GENERATED ALWAYS AS IDENTITY,
    name       TEXT      NOT NULL,
    CONSTRAINT pk_products PRIMARY KEY (product_id)
);
```

What PostgreSQL creates for a PK:

- An index on the PK column(s) — always a B-tree
- A NOT NULL constraint on each PK column
- Internally, a UNIQUE constraint

UNIQUE

```
-- Single column unique
CREATE TABLE users (
    email TEXT NOT NULL UNIQUE
);

-- Named unique constraint
CREATE TABLE users (
    email TEXT NOT NULL,
    CONSTRAINT uq_users_email UNIQUE (email)
);

-- Multi-column unique (combination must be unique)
CREATE TABLE employee_departments (
    employee_id INTEGER NOT NULL,
    department_id INTEGER NOT NULL,
    role        TEXT      NOT NULL,
    UNIQUE (employee_id, department_id) -- an employee can only be in a dept once
);

-- NULL handling: NULLs are NOT considered equal for UNIQUE
-- Multiple NULLs are allowed!
CREATE TABLE products (
    sku       TEXT UNIQUE,      -- allows multiple NULL skus
    barcode   TEXT UNIQUE      -- allows multiple NULL barcodes
);

INSERT INTO products (sku) VALUES (NULL);
INSERT INTO products (sku) VALUES (NULL); -- succeeds! two NULLs allowed
INSERT INTO products (sku) VALUES ('ABC');
INSERT INTO products (sku) VALUES ('ABC'); -- ERROR: duplicate key
```

UNIQUE NULLS NOT DISTINCT (PostgreSQL 15+)

```
-- Treat NULLs as equal for uniqueness purposes
CREATE TABLE profiles (
  user_id    INTEGER NOT NULL,
  phone      TEXT UNIQUE NULLS NOT DISTINCT -- at most one NULL phone allowed
);
```

Partial UNIQUE index as a constraint

```
-- UNIQUE constraint with a condition (must use index, not constraint syntax)
CREATE UNIQUE INDEX uq_active_email ON users (email) WHERE is_active = TRUE;
-- Multiple inactive users with same email allowed; active users must be unique
```

FOREIGN KEY

```
-- Basic foreign key
CREATE TABLE orders (
  order_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(user_id),
  total       NUMERIC(12,4) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Explicit syntax with name and options
CREATE TABLE order_items (
  item_id     BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id    BIGINT  NOT NULL,
  product_id  INTEGER NOT NULL,
  quantity    INTEGER NOT NULL CHECK (quantity > 0),
  unit_price  NUMERIC(10,4) NOT NULL CHECK (unit_price > 0),
  CONSTRAINT fk_order_items_order
    FOREIGN KEY (order_id)
    REFERENCES orders(order_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_order_items_product
    FOREIGN KEY (product_id)
    REFERENCES products(product_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);
```

Referential Actions

ON DELETE / ON UPDATE actions:

NO ACTION	Default. Raises error if referenced row exists (checked at end of statement/transaction)
-----------	--

RESTRICT	Like NO ACTION but checked immediately (cannot defer, even with DEFERRABLE)
CASCADE	Delete/update child rows automatically (ON DELETE CASCADE is very common)
SET NULL	Set FK column to NULL in child rows (child FK column must allow NULL)
SET DEFAULT	Set FK column to its default in child rows

```
-- Cascade example
CREATE TABLE posts (
  post_id BIGINT PRIMARY KEY,
  user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE
  -- Deleting a user automatically deletes all their posts
);

-- Set NULL example
CREATE TABLE orders (
  order_id BIGINT PRIMARY KEY,
  assigned_to INTEGER REFERENCES employees(employee_id) ON DELETE SET NULL
  -- If an employee is deleted, orders become unassigned (NULL)
);

-- Self-referencing FK (tree/hierarchy)
CREATE TABLE categories (
  category_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  parent_id BIGINT REFERENCES categories(category_id) ON DELETE SET NULL
);
```

Foreign key index rule

Always create an index on the FK column in the child table!

```
-- FK references parent by user_id
-- But without index on child table:
-- DELETE FROM users WHERE user_id = 1 ← scans entire orders table!
CREATE INDEX idx_orders_user_id ON orders (user_id);
CREATE INDEX idx_order_items_order_id ON order_items (order_id);
```

CHECK

```
-- Column-level CHECK
CREATE TABLE products (
  product_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```

    name      TEXT      NOT NULL CHECK (length(trim(name)) >= 2),
    price     NUMERIC(12,4) NOT NULL CHECK (price > 0),
    discount  NUMERIC(5,2) CHECK (discount BETWEEN 0 AND 100),
    stock     INTEGER NOT NULL CHECK (stock >= 0),
    rating    REAL      CHECK (rating >= 0 AND rating <= 5.0)
);

-- Table-level CHECK (can reference multiple columns)
CREATE TABLE bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    check_in   DATE      NOT NULL,
    check_out  DATE      NOT NULL,
    adults     SMALLINT NOT NULL CHECK (adults > 0),
    children   SMALLINT NOT NULL DEFAULT 0 CHECK (children >= 0),
    CONSTRAINT valid_dates CHECK (check_out > check_in),
    CONSTRAINT max_stay    CHECK (check_out - check_in <= 30),
    CONSTRAINT max_guests  CHECK (adults + children <= 10)
);

-- CHECK with regex
CREATE TABLE users (
    username TEXT NOT NULL CHECK (username ~ '^[a-zA-Z0-9_]{3,30}$'),
    email    TEXT NOT NULL CHECK (email ~ '^[^@\\s]+@[^@\\s]+\\. [^@\\s]+$',),
    phone    TEXT CHECK (phone IS NULL OR phone ~ '^\\+?[0-9\\s\\-\\(\\)]{7,20}$')
);

-- CHECK with NULL handling (NULL passes most checks)
CREATE TABLE test (
    val INTEGER CHECK (val > 0) -- NULL passes this check!
    -- NULL is neither > 0 nor NOT > 0; the CHECK expression is NULL, which is
    ignored
);
INSERT INTO test VALUES (NULL); -- succeeds
INSERT INTO test VALUES (5);   -- succeeds
INSERT INTO test VALUES (-1);  -- ERROR: check constraint violated

```

EXCLUSION Constraints

EXCLUSION constraints generalize UNIQUE to use arbitrary operators.

```

-- Prevent overlapping bookings using range types
CREATE TABLE room_bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    INTEGER NOT NULL,
    guest      TEXT     NOT NULL,
    period     DATERANGE NOT NULL,
    CONSTRAINT no_overlap EXCLUDE USING GIST (
        room_id WITH =, -- same room
        period  WITH && -- overlapping dates
    )
)

```

```

);

-- EXCLUDE with btree_gist (needed for non-geometric types with GIST)
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE employee_schedules (
    schedule_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id INTEGER NOT NULL,
    role TEXT NOT NULL,
    active_period daterange NOT NULL,
    CONSTRAINT one_role_at_a_time EXCLUDE USING GIST (
        employee_id WITH =,
        active_period WITH &&
    )
);

-- Exclusion with circles (no overlapping circular zones)
CREATE TABLE delivery_zones (
    zone_id SERIAL PRIMARY KEY,
    zone_name TEXT NOT NULL,
    area CIRCLE NOT NULL,
    CONSTRAINT no_zone_overlap EXCLUDE USING GIST (area WITH &&)
);

```

Deferrable Constraints

By default, constraints are checked after **each statement**. Deferrable constraints can be checked at **end of transaction**.

```

-- Declaring deferrable constraints
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    manager_id INTEGER REFERENCES employees(emp_id)
        DEFERRABLE INITIALLY DEFERRED -- checked at COMMIT, not per statement
);

-- INITIALLY IMMEDIATE: default behavior (check per statement), but can be deferred
-- INITIALLY DEFERRED: deferred by default, checked at COMMIT

-- Defer during a transaction
BEGIN;
SET CONSTRAINTS fk_employees_manager DEFERRED;
-- Now you can insert employees before their managers
INSERT INTO employees VALUES (1, 'Alice', 2); -- manager 2 doesn't exist yet
INSERT INTO employees VALUES (2, 'Bob', NULL); -- Bob is Alice's manager
COMMIT; -- FK check happens here, both rows exist → succeeds

-- Use case: circular FK relationships
-- Useful when loading data where A references B and B references A

```

Managing Constraints (ALTER TABLE)

```
-- Add constraint
ALTER TABLE products ADD CONSTRAINT chk_positive_price CHECK (price > 0);
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);
ALTER TABLE orders ADD CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(user_id);

-- Add NOT NULL (two steps for existing tables)
UPDATE users SET phone = '' WHERE phone IS NULL; -- fix NULLs first
ALTER TABLE users ALTER COLUMN phone SET NOT NULL;

-- Drop constraint
ALTER TABLE products DROP CONSTRAINT chk_positive_price;
ALTER TABLE users DROP CONSTRAINT uq_users_email;

-- Rename constraint
ALTER TABLE products RENAME CONSTRAINT old_name TO new_name;

-- Validate existing data before adding constraint (PostgreSQL 9.1+)
-- Step 1: Add NOT VALID (doesn't check existing rows, fast, no lock)
ALTER TABLE orders ADD CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    NOT VALID;

-- Step 2: Validate (scans existing rows, lighter lock than full rebuild)
ALTER TABLE orders VALIDATE CONSTRAINT fk_orders_user;
-- This approach minimizes lock time on large tables
```

Constraint Violations and Error Handling

```
-- PostgreSQL error codes for constraint violations:
-- 23502: not_null_violation
-- 23503: foreign_key_violation
-- 23505: unique_violation
-- 23514: check_violation
-- 23P01: exclusion_violation

-- Handle in PL/pgSQL
CREATE OR REPLACE FUNCTION safe_insert_user(p_email TEXT, p_name TEXT)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE
    v_user_id BIGINT;
BEGIN
    INSERT INTO users (email, full_name)
    VALUES (p_email, p_name)
    RETURNING user_id INTO v_user_id;
    RETURN v_user_id;
```



```

EXCEPTION
    WHEN unique_violation THEN
        -- Email already exists; return existing user_id
        SELECT user_id INTO v_user_id FROM users WHERE email = p_email;
        RETURN v_user_id;
    WHEN not_null_violation THEN
        RAISE EXCEPTION 'Email and name are required';
    WHEN check_violation THEN
        RAISE EXCEPTION 'Invalid email format: %', p_email;
END;
$$;

-- INSERT ... ON CONFLICT (upsert)
INSERT INTO users (email, full_name)
VALUES ('alice@example.com', 'Alice')
ON CONFLICT (email)
DO UPDATE SET full_name = EXCLUDED.full_name, updated_at = now();

-- INSERT ... ON CONFLICT DO NOTHING
INSERT INTO users (email, full_name)
VALUES ('alice@example.com', 'Alice')
ON CONFLICT (email) DO NOTHING;

```

SQL Examples

Fully constrained e-commerce schema

```

CREATE TABLE categories (
    category_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE CHECK (length(trim(name)) > 0),
    parent_id SMALLINT REFERENCES categories(category_id) ON DELETE SET NULL,
    CONSTRAINT no_self_reference CHECK (parent_id != category_id)
);

CREATE TABLE products (
    product_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sku TEXT NOT NULL UNIQUE CHECK (sku ~ '^[A-Z0-9-]{3,20}$'),
    name TEXT NOT NULL CHECK (length(trim(name)) >= 2),
    category_id SMALLINT NOT NULL REFERENCES categories(category_id),
    base_price NUMERIC(12,4) NOT NULL CHECK (base_price >= 0),
    sale_price NUMERIC(12,4) CHECK (sale_price >= 0 AND sale_price <=
base_price),
    stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT price_logic CHECK (
        sale_price IS NULL OR sale_price < base_price
    )
);

```

```

CREATE TABLE customers (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email       TEXT    NOT NULL UNIQUE CHECK (email ~ '^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$'),
  phone       TEXT    UNIQUE CHECK (phone IS NULL OR phone ~ '^[1-9][0-9]{7,14}$'),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE orders (
  order_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id  BIGINT NOT NULL REFERENCES customers(customer_id),
  status       TEXT    NOT NULL DEFAULT 'pending'
  CHECK (status IN
('pending', 'confirmed', 'shipped', 'delivered', 'cancelled')),
  total        NUMERIC(12,4) NOT NULL CHECK (total >= 0),
  ordered_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  shipped_at    TIMESTAMPTZ,
  CONSTRAINT shipped_after_ordered CHECK (
    shipped_at IS NULL OR shipped_at >= ordered_at
  )
);

CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_status  ON orders (status) WHERE status != 'delivered';

```

Common Mistakes

- Forgetting indexes on FK columns in child tables** — Every FK should have a corresponding index in the referencing (child) table.
- Using CHECK for multi-row or cross-table validation** — CHECK only works on the current row. For multi-table validation, use triggers or application logic.
- Assuming NULL fails a CHECK constraint**

```

CHECK (age > 0) -- NULL passes this! (NULL > 0 is NULL, not FALSE)
-- To enforce non-null AND positive:
age INTEGER NOT NULL CHECK (age > 0)

```

- Not naming constraints** — Unnamed constraints get auto-generated names. Error messages become cryptic. Always name constraints.
- ON DELETE CASCADE without consideration** — Cascading deletes can remove far more data than expected. Consider `SET NULL` or `RESTRICT`.
- Adding FK without NOT VALID on large tables** — Full table scan + strong lock. Use `NOT VALID` then `VALIDATE CONSTRAINT` separately.

Best Practices

1. **Name all constraints** using a consistent convention: `pk_<table>` , `fk_<table>_<ref>` , `uq_<table>_<col>` , `chk_<table>_<rule>` .
 2. **Always index FK columns** in child tables.
 3. **Think carefully about CASCADE** — document the intent with a comment.
 4. **Use NOT VALID + VALIDATE CONSTRAINT** when adding constraints to large live tables.
 5. **Use domain types** for frequently repeated CHECK patterns.
 6. **Use CHECK for domain validation** (positive numbers, valid enums, format validation) — don't rely solely on application code.
 7. **Use EXCLUSION constraints** instead of check-at-application-layer for overlap prevention.
-

Performance Considerations

- **PRIMARY KEY** creates a B-tree index automatically — free index for PK lookups.
- **UNIQUE** creates a B-tree index automatically — free index for unique lookups.
- **FK columns** in child tables need a manual index — not created automatically.
- **CHECK constraints** add minimal overhead (microseconds per row).
- **EXCLUSION constraints** require GIST index — more overhead than UNIQUE.
- **Adding constraints to existing tables:**

```
NOT NULL:    Full table scan + ACCESS EXCLUSIVE lock (fast)
CHECK:       Full table scan + ACCESS EXCLUSIVE lock
UNIQUE:      Creates index (like CREATE INDEX) – long, but can be CONCURRENT
FK NOT VALID: Fast, minimal locking
FK VALIDATE: Full scan, SHARE ROW EXCLUSIVE lock
```

```
-- Zero-downtime constraint addition strategy for large tables:
-- 1. Add the constraint as NOT VALID (fast, brief lock)
ALTER TABLE orders ADD CONSTRAINT fk_user
    FOREIGN KEY (user_id) REFERENCES users(user_id) NOT VALID;

-- 2. Validate concurrently (longer, but lighter lock)
ALTER TABLE orders VALIDATE CONSTRAINT fk_user;
-- During step 2, reads and writes to the table are still allowed
```

Interview Questions & Answers

Q1: What is the difference between PRIMARY KEY and UNIQUE constraints?

A: PRIMARY KEY = UNIQUE + NOT NULL. A table can have only one PRIMARY KEY but multiple UNIQUE constraints. NULL values are allowed in UNIQUE columns (multiple NULLs are permitted since NULLs are not equal to each other). A primary key is the canonical row identifier.

Q2: Does a FOREIGN KEY constraint create an index?

A: No. PostgreSQL creates an index for the PRIMARY KEY and UNIQUE constraint columns, but NOT for FK columns. You must manually create an index on the FK column in the child table. Without it, deleting from the parent table causes a full scan of the child table.

Q3: What is the difference between `ON DELETE CASCADE` and `ON DELETE SET NULL` ?

A: `CASCADE` deletes the child row when the parent is deleted — useful for dependent entities (order items when order is deleted). `SET NULL` sets the FK column to `NULL` — useful for optional relationships (orders assigned to employees; when an employee is deleted, the order remains but becomes unassigned).

Q4: Why doesn't a `NULL` value fail a `CHECK` constraint like `CHECK (age > 0)` ?

A: Because `NULL > 0` evaluates to `NULL`, not `FALSE`. PostgreSQL only rejects a row if the `CHECK` expression evaluates to `FALSE`. `NULL` (unknown) is treated as "we can't tell, so we allow it." Add a separate `NOT NULL` constraint if you want to prevent `NULL`s.

Q5: What is a deferrable constraint and when would you use it?

A: A deferrable constraint can have its check postponed to the end of the transaction (`COMMIT` time) instead of after each statement. Useful for: (1) circular FK references where A references B and B references A, (2) bulk data loads where you insert many rows that collectively satisfy constraints but individual inserts might temporarily violate them, (3) reordering rows that have a unique ordering column.

Q6: How would you add a `NOT NULL` constraint to a large production table with minimal downtime?

A: (1) Set a `DEFAULT` value if needed. (2) Backfill existing `NULL` rows: `UPDATE table SET col = 'default' WHERE col IS NULL`. (3) Add the constraint: this still requires a lock but is fast since no existing `NULL`s remain. Alternatively, in PostgreSQL 11+: just `ALTER COLUMN SET NOT NULL` — PostgreSQL is smart enough to skip the scan if a suitable `CHECK` constraint already exists or via constraint validation metadata.

Q7: What is the `NOT VALID` option for constraints?

A: `NOT VALID` adds the constraint to the table but does not validate existing rows — only new/updated rows are checked. This is fast and uses a brief lock. You then run `ALTER TABLE VALIDATE CONSTRAINT name` to verify existing data, which uses a lighter lock. This is the recommended approach for adding FK constraints to large production tables.

Q8: Can you put a `FOREIGN KEY` on an array column?

A: No. PostgreSQL does not support FK constraints that reference elements within arrays. This is a fundamental limitation of array-based design. If you need FK integrity on a collection of references (e.g., a list of user IDs), you must use a normalized junction table.

Q9: What happens to `CHECK` constraints when you `ALTER TABLE ... ALTER COLUMN TYPE` ?

A: The `CHECK` constraint is preserved but may need to be verified still evaluates correctly with the new type. PostgreSQL will validate the constraint with the new type. If it fails, the `ALTER` fails and you need to modify the constraint first.

Q10: How does the `EXCLUSION` constraint differ from a `UNIQUE` constraint?

A: `UNIQUE` checks if two rows have identical values (equality). `EXCLUSION` allows you to specify any operator — the most common being `&&` (overlap) for range types. `EXCLUSION` says "no two rows can have values where the specified operator returns `TRUE`." This enables preventing overlapping bookings, conflicting schedules, or overlapping circular areas.

Exercises with Solutions

Exercise 1

Design a `subscriptions` table with an EXCLUSION constraint preventing overlapping active subscription periods for the same user.

Solution:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE subscriptions (
    sub_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id     INTEGER NOT NULL,
    plan        TEXT NOT NULL CHECK (plan IN ('basic', 'pro', 'enterprise')),
    period      DATERANGE NOT NULL,
    price_paid  NUMERIC(10,2) NOT NULL CHECK (price_paid >= 0),
    CONSTRAINT no_overlapping_subs
        EXCLUDE USING GIST (user_id WITH =, period WITH &&),
    CONSTRAINT valid_period CHECK (NOT isempty(period))
);
```

Exercise 2

Add constraints to a `bank_transfers` table to enforce: amount > 0, from_account ≠ to_account, and status is a valid value. Use named constraints.

Solution:

```
CREATE TABLE bank_transfers (
    transfer_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    from_account_id INTEGER NOT NULL,
    to_account_id  INTEGER NOT NULL,
    amount         NUMERIC(15,4) NOT NULL,
    currency       CHAR(3) NOT NULL DEFAULT 'USD',
    status         TEXT NOT NULL DEFAULT 'pending',
    initiated_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at   TIMESTAMPTZ,
    CONSTRAINT chk_positive_amount CHECK (amount > 0),
    CONSTRAINT chk_different_accounts CHECK (from_account_id != to_account_id),
    CONSTRAINT chk_valid_status CHECK (
        status IN ('pending', 'processing', 'completed', 'failed', 'reversed')
    ),
    CONSTRAINT chk_valid_currency CHECK (currency ~ '^[A-Z]{3}$'),
    CONSTRAINT chk_completion_order CHECK (
        completed_at IS NULL OR completed_at >= initiated_at
    )
);
```

Cross-References

- `09_sequences_identity.md` — sequences used by identity PKs
- `10_schemas_namespacing.md` — constraint naming across schemas
- `../07_Indexes/` — indexes created by constraints

- `../09_Transactions_Concurrency/` — deferrable constraints in transactions
- `../06_Database_Design/` — constraint design patterns in schema files